

The Dedekind Topos: A Structuralist Mereology in C++23

The Dedekind Authors

March 27, 2026

Abstract

Your abstract text.

1 Introduction

The quest for a unified computational environment that simultaneously satisfies the rigour of formal proof assistants and the raw efficiency of hardware-level numerics remains a central challenge in computer science. Historically, this has been a binary choice: one either employs a dedicated prover such as Coq or Lean to ensure mathematical correctness at the cost of execution speed, or one utilizes traditional numeric libraries that prioritize throughput while treating mathematical axioms as mere documentation.

In this paper, we propose **dedekind**, a Domain Specific Language (DSL) embedded in modern C++23, designed to bridge this dichotomy. Our approach is rooted in the *Curry-Howard correspondence*, which establishes a direct isomorphism between computer programs and mathematical proofs. By leveraging C++23 **concepts** and template metaprogramming, we treat the C++ type system as a logic engine. While the Clang compiler may not be the most sophisticated theorem prover in existence, its ubiquity and industry-standard optimization passes make it perhaps the most accessible vehicle for what we term *Computational Structuralism*.

Following the intuition provided by Wadler in *Theorems for Free*, **dedekind** utilizes parametric polymorphism to derive structural properties of mathematical objects—such as associativity, commutativity, and idempotency—directly from their type signatures. By encoding mereotopological and category-theoretic axioms into the "structural registry" of the C++ translation unit, we enable the compiler to perform symbolic pruning of the computational Directed Acyclic Graph (DAG) before a single machine instruction is emitted. The resulting binary is as lean as a manually optimized "naked" loop, yet remains as rigorous as a formal proof, reifying the DAG as a concrete artifact of the underlying mathematical theorem.

While the Clang compiler is not traditionally viewed as a formal verification tool, we embrace its role as a "poor man's theorem prover" as a central pillar of our methodology. We view the "hacking" of industry-standard optimization passes to resolve structural identities at compile-time not as a workaround, but as a strategic alignment with the most pervasive toolchain in modern computing. This approach allows us to leverage the ubiquity of the C++ ecosystem to theorems for free in a mainstream environment. As an added benefit, the resulting binary is as lean as a manually optimized "naked" loop, while being validated by rigorous and formal proofs.

2 Methods

The methodology of **dedekind** is predicated on the translation of formal mathematical structures into a stratified registry of C++23 modules. This stratification allows for a "bottom-up" synthesis of complex mathematical objects, where each level serves as a verified foundation for the next.

2.1 The Unified Dedekind Ontology

The `dedekind.ontology` module serves as a central registry of structural truths. It aggregates skeletal, mereological, and algebraic laws to enable the synthesis of the *Continuum* from the *Discrete*. By adopting a *Structuralist* posture—viewing mathematical objects as positions within a structure rather than isolated collections—we facilitate zero-overhead formal verification within the C++ type system.

The ontology is organized into four primary levels of abstraction, mirroring the historical construction of mathematics:

Level -1 to 0: Foundations (Bricks and Cement) At the base lies `:logic`, defining the subobject classifier Ω and predicate logic. This is supported by `:species`, which provides reified machine primitives, and `:category`, which establishes the "Highways" of the system via Morphisms, Functors, and Kleisli Triples.

Level 1: Space (Body and Relation) The `:mereology` partition defines the rules of presence (parts and wholes), while `:order` and `:topology` establish the rules of relation and closeness (posets, lattices, and open sets).

Level 2: Scale (Measurement and Path) The `:sequences` module maps magnitudes to enumerations, providing the necessary machinery for limits and convergence.

Level 3: Soul (Harmony and Action) This level implements the pure algebraic laws (`:algebra`) including Groups, Rings, and Fields, alongside their realized `:morphologies` (Cyclic, Ordered, and Archimedean forms).

Level 4: Realization (The Registry) The final level, `:geometry` and `:numbers`, realizes the continuum through the species registry, formally constructing the inclusion $\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R}$.

2.2 The Semantic Mapping

Crucially, the `dedekind.ontology` acts as the definitive mapping between formal mathematical nomenclature and the concrete syntax of the C++23 language. Rather than introducing a foreign vocabulary, we anchor established axioms directly into the standard operator set and primitive types. For instance, the mereological *Sum* and *Product* are reified as the bitwise operators `|` and `&`, while the *Mereological Complement* (the remainder) is bound to the negation operator `!`.

By establishing this explicit correspondence—where `operator<=` serves as the formal proof of *Parthood* ($\sqsubseteq$) and `operator/` represents the *Quotienting* of a species—we ensure that the resulting code is not merely a simulation of a mathematical model, but a direct execution of it. This semantic alignment allows the programmer to write expressions that are readable as equations, while the compiler treats them as a sequence of verifiable type-transformations. In this architecture, the C++ `concept` becomes the "Gatekeeper of Truth," ensuring that a species cannot participate in an operation (such as a Join-Semilattice union) unless it has first provided a manual or structural certification of its algebraic invariants.

2.3 The Registry of Structural Proofs

The transition from a raw computational graph to a simplified mathematical identity requires a formal "Seal of Truth." In `dedekind`, we implement a *Traits Registry* that forces the developer to explicitly certify the equational axioms of a species before it can participate in the lattice-based optimizations of the dispatcher.

This is achieved through a dual-layered system of **Trait Ledgers** and **Discovery Concepts**. For each fundamental algebraic law, we define a primary template (the ledger) which defaults to `false`:

Table 1: The Dedekind Semantic Mapping: A Registry of Formal Correspondences

Mathematical Concept	Symbol	C++ Operator / Type	C++ Concept	Basis
<i>Category Theory</i>				
Kleisli Extension (The Push)	$(T, \eta, \gg=)$	<code>operator>=</code>	<code>IsKleisliExtension</code>	
Co-Kleisli Extension (The Pull)	$(W, \varepsilon, \ll=)$	<code>operator<=</code>	<code>IsCoKleisliExtension</code>	
Morphic Injection (Unit)	η	<code>into<F> / singleton</code>	<code>IsMonad</code>	$\eta : T \rightarrow F(T)$
Morphic Extraction (Counit)	ε	<code>extract<F></code>	<code>IsComonad</code>	$\varepsilon : F(T) \rightarrow T$
Characteristic Morphism	χ	<code>operator()</code>	<code>IsCharacteristic</code>	
Kleisli Composition	$\circ_K$	<code>operator></code>	<code>IsMonad</code>	
Co-Kleisli Composition	$\circ_{CoK}$	<code>operator<</code>	<code>IsComonad</code>	
Counit (Extraction)	ε	<code>extract<F></code>	<code>IsComonad</code>	
<i>Mereology</i>				
Proper Parthood	χ	<code>operator()</code>	<code>IsProperPart</code>	<code>IsCharacteristic</code>
Parthood (Pre-order)	$\sqsubseteq$	<code>operator<=</code>	<code>IsPartOf</code>	
Mereological Sum (Join)	$\sqcup$	<code>operator </code>	<code>IsJoinSemilattice</code>	
Mereological Product (Meet)	$\sqcap$	<code>operator&</code>	<code>IsMeetSemilattice</code>	
Universal Complement	$\neg$	<code>operator!</code>	<code>IsComplemented</code>	
Initial Object (Empty)	$\emptyset$	<code>\emptyset<T></code>	<code>IsInitialObject</code>	
Terminal Object (Universal)	Ω	<code>\Omega<T></code>	<code>IsTerminalObject</code>	
<i>Set Theory</i>				
Membership	$\in$	<code>operator()</code>	<code>IsSet</code>	<code>IsProperPart</code>
Subset	$\subseteq$	<code>operator<=</code>	<code>IsSubset</code>	<code>IsPartOf</code>
Countable Infinity	$\aleph_0$	<code>\aleph_0</code>	<code>IsCardinality</code>	
Continuum Cardinality	$\beth_1$	<code>\beth_1</code>	<code>IsCardinality</code>	
<i>Order Theory</i>				
Pre-order	$\sqsubseteq$	<code>operator<=</code>	<code>IsPartiallyOrdered</code>	<code>IsPartOf</code>
<i>Algebra</i>				
Structural Origin (Zero)	0	<code>T::origin()</code>	<code>IsPointed</code>	
Additive Composition	$+$	<code>operator+</code>	<code>IsGroup</code>	
Additive Inverse	$-$	<code>operator-</code>	<code>IsGroup</code>	
Multiplicative Composition	$\times$	<code>operator*</code>	<code>IsRing</code>	
Multiplicative Inverse	$/, ^{-1}$	<code>operator/</code>	<code>IsField</code>	
Associativity	$(a \circ b) \circ c$	<code>is_associative_v</code>	<code>IsSemigroupoid</code>	
Idempotency	$a \circ a = a$	<code>is_idempotent_v</code>	<code>IsIdempotent</code>	
Magnitude / Count	$ S $	<code>s.size() / s.cardinality()</code>	<code>IsCardinality</code>	
Supremum / Infimum	$\sup, \inf$	<code>s.supremum(), s.infimum()</code>	<code>HasExtrema</code>	

Listing 1: The Associativity Ledger in :species

```

export template <typename T, typename Op>
struct is_associative : std::false_type {};

export template <typename T, typename Op>
inline constexpr bool is_associative_v = is_associative<T, Op>::value
;

```

A species S is only recognized as a `IsSemigroupoid` if it (or the ontology) provides a specialization of this ledger. To allow for "Interoperable Discovery," we provide a fallback mechanism that probes the species for internal certification:

Listing 2: The Concept of Formal Association

```

export template <typename T, typename Op>
concept IsAssociative = IsMagmoid<T, Op> && requires {
    requires is_associative_v<T, Op>;
}

```

```
};

// Discovery Fallback: Probing the Species for an Intrinsic Proof
template <typename T, typename Op>
  requires requires { T::is_associative_for<Op>; }
struct is_associative<T, Op> : T::template is_associative_for<Op> {};
```

By anchoring concepts like `IsJoinSemilattice` in these requirements (requiring both `IsAssociative` and `IsIdempotent`), we ensure that the *Lattice Dispatcher* never performs a "speculative" pruning. If the developer has not provided the proof, the dispatcher falls back to a safe, unoptimized `UnionSet` synthesis. This "Axiom of Responsibility" ensures that the performance of the "Naked Loop" is always mathematically justified by a prior formal certification.

2.4 Reverse Curry-Howard Validation

To verify the integrity of the proposed mapping between formal axioms and C++ concepts, we employ a "Reverse Curry-Howard" validation strategy. Rather than simply assuming the correctness of our ontological translations, we use the inherent properties of C++ built-in types as an empirical proof of the reverse-engineered axioms.

By executing a suite of `static_assert` evaluations on primitive types—such as verifying that `bool` satisfies the requirements of an `IsJoinSemilattice` or that bitwise operations on `uint32_t` adhere to the `IsMereologicalLattice` consistency axioms—we confirm that the machine-level behavior is isomorphic to the formal mathematical theory. In this posture, the built-in types serve as the "Initial Models" for our concepts. If the compiler successfully verifies that a standard integer under bitwise conjunction satisfies our `IsMeetSemilattice` proof, it provides an automated, formal confirmation that our C++ concepts correctly capture the essential structural invariants found in the mathematical literature.

3 Implementation: The Functor Highway

The operational efficiency of `dedekind` is achieved through a technique we term *Synthetic Inference*. This level represents the "Skeletal Foundation" of the ontology, where machine-level instructions are unified with the abstract laws of Category Theory. By treating the C++ type system as a formal proof assistant, we ensure structural integrity via compile-time verification, or *Static Mereology*.

3.1 The Action-First Bootstrapping Strategy

In textbook Category Theory, a Monad is traditionally defined as a *Monoid in the category of Endofunctors*. However, the structural implementation of this definition in C++ faces significant language constraints:

1. **Partial Specialization Limitations:** C++ forbids the partial specialization of function templates (e.g., `fmap<F>`), preventing a centralized, generic definition for disparate species.
2. **ADL Resolution:** Function templates called via explicit name-lookup cannot trigger Argument-Dependent Lookup (ADL), making it difficult for the ontology to "discover" mappings for types defined in downstream modules.

To resolve this circular dependency, `dedekind` implements an *Action-First* bootstrapping strategy. By defining Monads and Comonads as *Extension Systems* (Kleisli Triples consisting of η/ϵ and $\gg= / \ll=$), we utilize operator overloading on concrete objects. This triggers ADL at the call site, allowing the Level 0 foundation to instantiate a derived `fmap` without prior knowledge of Level 1 species.

3.2 The Unified Highway Bridge

The `fmap` dispatcher serves as the implementation of this strategy. It automates the derivation of functorial mappings from the underlying monadic "Push" or comonadic "Pull."

Listing 3: The Unified Highway Bridge (`fmap` derivation)

```
export template <template <typename...> typename F, typename Arrow,
                typename T = typename Arrow::Domain,
                typename U = typename Arrow::Codomain>
requires IsArrow<Arrow, T, U>
constexpr IsArrow<F<T>, F<U>> auto fmap(Arrow f) {
  if constexpr (std::same_as<F<T>, T>) {
    return f; // Identity Functor
  } else if constexpr (IsKleisliExtension<F, T, U>) {
    /** @theorem fmap(f) = m >>= (eta o f) */
    return arrow<F<T>, F<U>>([f](const F<T>& m) {
      return m >>= [f](const T& x) { return into<F, U>(f(x)); });
    });
  } else if constexpr (IsCoKleisliExtension<F, T, U>) {
    /** @theorem fmap(f) = w <<= (f o epsilon) */
    return arrow<F<T>, F<U>>([f](const F<T>& w) {
      return w <<= [f](const F<T>& ctx) { return f(extract<F, T>(ctx)
        ); });
    });
  } else {
    static_assert(sizeof(T) == 0, "Ontology_Error:_Species_lacks_
      highway_structure.");
  }
}
```

3.3 Highway Notation: Directional Vectors

To facilitate a readable "Algebra of Programming," we map categorical data flows to directional operators, creating a "Highway Notation" that reflects the vector of computation across the ontology:

- `value » into<F>` : Represents η (Unit), lifting a species into a context.
- `box « extract<F>` : Represents ε (Counit), sampling a species from a context.
- `box »= f` : Represents *Bind*, the forward monadic composition (Left-to-Right).
- `box <= f` : Represents *Extend*, the contextual comonadic composition (Right-to-Left).

This mapping ensures that the "Categorical Cement" holding the "Algebraic Bricks" together is both syntactically expressive and computationally invisible.